StaffSync

Modern Human Resource Management Solution

Project Documentation

Project	StaffSync - MERN Stack HRM Web Application
Domain	Information Technology
Project Type	Individual
Submitted By	Neha Patidar
Enrollment No.	EN24CS3010683
Technologies	MongoDB, Express.js, React.js, Node.js, JWT, Bootstrap

Table of Contents

1. Introduction
2. Objectives
3. Scope of the Project
4. Technology Stack
5. System Requirements
6. System Architecture
7. User Roles and Modules
8. Database Design
9. REST API Endpoints
10. Project Structure
11. Installation and Setup
12. Testing
13. Security Features
14. Limitations and Future Enhancements
15. Conclusion
16. Project Links

1. Introduction

StaffSync is a comprehensive web-based Human Resource Management (HRM) application built using the MERN Stack (MongoDB, Express.js, React.js and Node.js). It streamlines and automates the core HR processes of an organization such as employee record keeping, leave management, attendance tracking, company announcements and training programs.

The system provides three distinct roles - **HR**, **Manager** and **Employee** - each with its own access level, ensuring efficient management and transparency across all operations. By digitizing these functions on a single centralized platform, StaffSync improves productivity, communication and workflow between departments.

2. Objectives

- Digitize and centralize employee data and HR operations.
- Provide secure, role-based access for HR, Managers and Employees using JWT authentication.
- Automate the leave application and approval workflow.
- Enable employees to mark daily attendance (check-in / check-out) and let HR and managers monitor it.
- Allow HR to publish company announcements and news to all employees.
- Allow managers to create and assign training and development programs to their team members.
- Build a responsive, easy-to-use interface that works on desktop and mobile browsers.

3. Scope of the Project

The project covers the complete lifecycle of basic HR operations inside an organization: user registration and login, employee management, leave requests and approvals, attendance, announcements and training management. Payroll, recruitment and performance appraisal are outside the current scope and are listed under future enhancements.

4. Technology Stack

Layer	Technology	Purpose
Frontend	React.js (Vite), Bootstrap 5, Axios	Responsive user interface and API calls
Backend	Node.js, Express.js	REST API, business logic, routing
Database	MongoDB with Mongoose	Storage of users, leaves, attendance, announcements, training
Authentication	JSON Web Token (JWT), bcryptjs	Secure login, password hashing, role-based access
Tools	VS Code, Postman, Git and GitHub, MongoDB Atlas	Development, API testing, version control, cloud DB

5. System Requirements

Software Requirements

Software	Requirement
Operating System	Windows 10/11, macOS or Linux

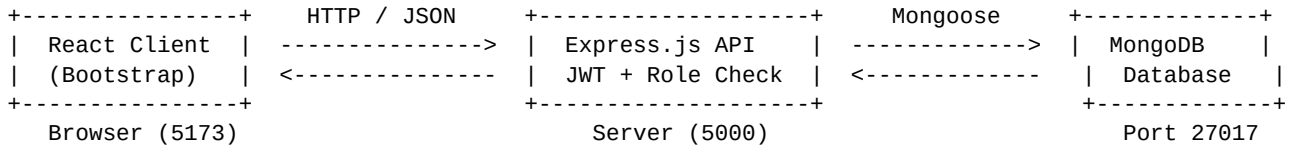
Software	Requirement
Node.js	v16 or above
npm	v8 or above
MongoDB	Community Server (local) or MongoDB Atlas (cloud)
Browser	Latest Google Chrome / Firefox
Editor and Tools	Visual Studio Code, Postman, Git

Hardware Requirements

Component	Minimum / Recommended
Processor	Intel Core i5 (8th Gen) / AMD Ryzen 5 or better
RAM	8 GB minimum (16 GB recommended)
Storage	At least 1 GB free space
Display	1366 x 768 or higher

6. System Architecture

StaffSync follows a three-tier client-server architecture. The React frontend sends HTTP requests (with a JWT in the Authorization header) to the Express REST API. The API validates the token and role, performs the business logic and reads or writes data in MongoDB through Mongoose models.



Request flow: Login returns a JWT. The frontend stores it and attaches it to every request. The *protect* middleware verifies the token and the *allow* middleware checks that the user's role may access the route.

7. User Roles and Modules

Access Matrix

Module	Employee	Manager	HR
Profile / Employees	View and edit own profile	View own team members	Add, view and delete all employees
Leave Management	Apply and view own history	Approve / reject team leaves	Approve / reject all leaves
Attendance	Check-in / check-out, own history	View team attendance	View all attendance
Announcements	View	View	Post and delete
Training and Development	View assigned programs	Create, assign and delete programs	View all programs

Module Description

- **Authentication:** Register and login with email and password. Passwords are hashed and a JWT valid for one day is issued.
- **Employee Management:** HR creates employee accounts with role, department, designation and reporting manager. Every user can update their own profile.
- **Leave Management:** Employees apply with leave type, dates and reason. Status starts as *pending* and is changed to *approved* or *rejected* by the manager or HR.
- **Attendance:** One record per employee per day with check-in and check-out time. Duplicate check-in on the same day is blocked.
- **Announcements:** HR posts company news with a title and message, visible to everyone.
- **Training:** Managers create programs with a title, date and description and assign them to team members.

8. Database Design

MongoDB collections (Mongoose models) used in the project:

Collection	Fields
User	name, email (unique), password (hashed), role (hr / manager / employee), department, designation, phone, manager (ref User), timestamps
Leave	employee (ref User), type (Casual / Sick / Earned / Other), fromDate, toDate, reason, status (pending / approved / rejected), timestamps
Attendance	employee (ref User), date (YYYY-MM-DD), checkIn, checkOut, status, timestamps
Announcement	title, message, postedBy (ref User), timestamps
Training	title, description, date, assignedTo [ref User], createdBy (ref User), timestamps

Relationships: A manager is linked to many employees through the *manager* field of User. Leave, Attendance and Training documents reference the User collection by ObjectId.

9. REST API Endpoints

Base URL: `http://localhost:5000/api`. All routes except register and login need a Bearer token.

Method	Endpoint	Access	Description
POST	/auth/register	Public	Create a new user
POST	/auth/login	Public	Login and get JWT
GET	/auth/me	All	Current user details
GET / PUT	/employees/me	All	View / update own profile
GET	/employees	HR, Manager	List employees (manager sees own team)
POST	/employees	HR	Add employee
PUT / DELETE	/employees/:id	HR	Update / delete employee
POST	/leaves	All	Apply for leave
GET	/leaves/mine	All	Own leave history
GET	/leaves	HR, Manager	All / team leave requests
PUT	/leaves/:id/status	HR, Manager	Approve or reject a leave

Method	Endpoint	Access	Description
POST	/attendance/checkin	All	Mark check-in
POST	/attendance/checkout	All	Mark check-out
GET	/attendance/mine	All	Own attendance
GET	/attendance	HR, Manager	All / team attendance
GET	/announcements	All	List announcements
POST / DELETE	/announcements	HR	Post / delete announcement
GET	/training	All	Role-based training list
POST / DELETE	/training	Manager	Create / delete training

10. Project Structure

```
staffsync/
|-- backend/
|   |-- models/        User.js, Leave.js, Attendance.js, Announcement.js, Training.js
|   |-- routes/        auth.js, employees.js, leaves.js, attendance.js,
|   |                  announcements.js, training.js
|   |-- middleware/    auth.js  (protect, allow)
|   |-- server.js      Express app and MongoDB connection
|   |-- seed.js        Creates demo HR, Manager and Employee users
|   |-- .env           PORT, MONGO_URI, JWT_SECRET
|   `-- package.json
|-- frontend/
|   |-- src/
|   |   |-- App.jsx         Login / Register
|   |   |-- Dashboard.jsx   Navbar and role-based tabs
|   |   |-- Panels.jsx      Profile, Leaves, Attendance, Announcements,
|   |   |                   Training, Employees
|   |   |-- api.js          Axios instance with token interceptor
|   |   `-- main.jsx
|   |-- index.html, vite.config.js, package.json
`-- README.md
```

11. Installation and Setup

- Install Node.js (LTS), Git, MongoDB Community Server (or create a MongoDB Atlas cluster), VS Code and Postman.
- Open the `staffsync` folder in VS Code.

Backend

```
cd backend
npm install
npm run seed     # creates demo users
npm run dev      # runs on http://localhost:5000
```

Frontend (new terminal)

```
cd frontend
npm install
npm run dev      # runs on http://localhost:5173
```

Environment variables (backend/.env)

```
PORT=5000
MONGO_URI=mongodb://127.0.0.1:27017/staffsync
JWT_SECRET=your_secret_key
```

Demo credentials (password: 123456)

Role	Email
HR	hr@test.com
Manager	manager@test.com
Employee	employee@test.com

12. Testing

APIs were tested with Postman and the complete flow was verified from the user interface using all three roles.

No.	Test Case	Expected Result
1	Register a new user	User created and token returned
2	Login with wrong password	Error: Invalid credentials
3	Access protected route without token	401 - No token
4	Employee opens HR-only route (add employee)	403 - Access denied
5	Employee applies for leave	Leave saved with status pending
6	Leave with To date earlier than From date	Validation error shown
7	Manager approves a team member's leave	Status changes to approved
8	Manager tries to approve leave of another team	403 - Not your team member
9	Employee checks in twice on the same day	Error: Already checked in today
10	HR posts an announcement	Visible to all roles
11	Manager creates and assigns training	Visible to the assigned employees

13. Security Features

- Passwords are hashed with bcrypt before being stored.
- JWT-based authentication with token expiry of one day.
- Role-based authorization middleware on every protected route.
- Managers can act only on leave requests of their own team.
- Secrets (database URL and JWT secret) are kept in a .env file that is excluded from Git.

14. Limitations and Future Enhancements

- Payroll and salary slip generation.
- Leave balance and holiday calendar.
- Email or in-app notifications for leave status and announcements.

- Profile photo and document upload.
- Reports and charts for attendance and leave analytics.
- Password reset and email verification.
- Recruitment and performance review modules.

15. Conclusion

StaffSync successfully demonstrates a full-stack MERN application that digitizes the main HR processes of an organization. Role-based dashboards for HR, Managers and Employees, secure JWT authentication and a clean REST API make the system reliable and easy to extend. The project strengthened practical knowledge of React, Node.js, Express, MongoDB, asynchronous JavaScript and web security, and it can be further developed into a complete HR suite.

16. Project Links

Item	Link
GitHub Repository	https://github.com/en24cs3010683neha/staffsync
Live Demo	(add after deployment)